[Paper Critics 3] Accelerating Write by Exploiting PCM Asymmetries

1. Overview

   This paper utilizes the dual asymmetry of PCM by fast writing in 0 but power-intensive and slow writing in 1 but power-efficient to reduce the write time of a single cache line from 3440ns to 1260ns and results in an average system execution time reduction of 33.9%.

   PCM has an advantage on scalability, low leakage current and refresh-free operation. It is considered the successor to DRAM. However, slow writing is one of two main weaknesses of it. The author raises two approaches to solve them. First one is two-stage-write, which after written all 0, using times of parallelization to write all 1. Second one is two-stage-write-inv, which is based on first one, inversing the bits to ensure that the number of 1 in 16bits won't exceed half of bits which results in the parallelization would become twice.

   The paper uses M5 + DRAMsim to simulate the 4-core system and 13 groups of SPEC CPU 2006 workload. The main result compares to the representative method at that time, Flip-N-Write, the inv version would reduce the reading latency by 16.3%, and the execution time would improve by 9.2% performance.

2. Problem Definition

   The traditional PCM controller would schedule all bits by using time for writing-1 and current for writing-0 in parallel. It would use the worst scheduling plan of all writing tasks which do not exist in physics, waste time and power both.

   This issue would come from the physical properties of PCM unit. Writing 0 is called RESET, which requires melting the GST material with a large current for a short time and then rapidly cooling it to make it into a high-resistance amorphous state; writing 1 is called SET, which involves heating with a smaller current for a long time to allow the material to slowly crystallize into a low-resistance state. Because crystallization itself takes time, and process variations make that it is necessary to iterate programming followed by verification, writing a 1 takes much longer than writing a 0. In the parameters used here, writing a 0 takes about 50ns, and writing a 1 takes about 430ns, with a time ratio K of 9l the current required to write a 0 is twice that required to write a 1, with a current ratio L of 2.

   On the other side, PCM needs the writing voltage, which is more than Vdd, which comes from the power supply in internal Dickson charge pump in chip. The efficiency of charge pump is low, and the power cord noise is high, it couldn't provide a high instantaneous current, so chip could only write 2, 4, 8 or 16 bits at a time; this method is called write division mode. In the 8 bytes write unit, a cache line in 65 bytes must be written 8 times in sequence. Under these two restrictions, the traditional controller takes two assumptions for the safety issues, it makes all

write unit to write for writing long time of 1, let the 0 bits wait for these. At the same time, using the larger current for writing of 0 to estimate the consumption of all bits. It would lead to the bits that could write more 1 in parallel be restricted. However, no bits would slow and consume a lot of energy at the same time, these two assumptions seem to be over-conservative. After lengthened writing times result in longer bank occupancy, forcing read requests to queue and ultimately slowing down the entire system. Therefore, this paper aims to reduce the writing service time of a cache line without increasing power costs. The authors explicitly limit the focus to writing speed for SLC PCM, ignoring write endurance and MLC.

3. Methodology

   The approaches would be divided into two stages in this paper. Two-stage-write would separate the writing in 0 and 1, taking advantage of the asymmetry of time and current respectively; two-stage-write-inv then uses inversion encoding to overcome the speedup limit caused by the current ratio L.

   Two-stage-write, traditional way to mix 0 and 1 for anytime in writing, so only could use the worst case of scheduling. Two-stage-write makes it into 2 phases to write a cache line. The first phase only writes 0, the number of the written bits is same as traditional way but only need to wait for writing 0 in 50ns which is not the 430ns for writing 1. It makes it would quickly finish the writing in 8 times. The second phase only write 1 because the current in write 1 is only in half of 0, the same budget of electric energy consumption would make twice the writing bits from 16bits to 32bits, and the number of needed writings reduces from 8 to 4. Because the bit value written in each stage is the same, the controller can accurately control power consumption without needing to make conservative estimates. The overall write time is reduced from the traditional 3440ns to 2120ns, about 60% of the original. The authors also point out that this benefit depends only on the time difference and current difference between writing 0 and writing 1 and is unrelated to how many bits the chip can write at once. However, the speedup can only reach a multiple of the current difference, and the writing 1 stage remains the bottleneck.

   The second stage, two-stage-write-inv, in order to overcome the bottleneck from stage 1, the author adds bit-inverse. Counting the number of 1 in 16bits before writing, if exceeds 8 bits, then inverse all bits and write back and use an external flag bit to record. Repair by the flag in reading. Therefore, we could ensure that all 1-bits which needed to be written couldn’t exceed half of bits. The written bits could twice to 64bits in the first stage and the number of writing times could reduce from 4 to 2 times. The overall writing time could reduce to 1260ns. This

difference between the inverse and Flip-N-Write is only looking for new data itself and doesn’t need to compare the old data before writing.

The above logic uses a finite state machine within the PCM chip to control the current stage and the area to be written, and simple logic gates determine which round each bit is written in; the inversion version adds a counter to determine whether to invert. These circuits only modify the write path within the chip, without requiring modifications to the processor. The authors simulated a workload using a 4-core system with 13 SPEC CPU 2006 sets and compared the improvements in read latency and execution time with traditional write, Write Cancellation, and Flip-N-Write methods.

4. Pros and Cons

   Pros: This paper utilizes two asymmetries: fast writing of 0s and power-saving writing of 1s. The 0-writing phase saves the time previously required to write 1s, while the 1-writing phase uses previously wasted power margin to write more bits at once. This allows it to reduce writing time to about one-third of the traditional approach without increasing power budget, write throughput, or requiring reading old data beforehand like Flip-N-Write. Furthermore, the required modifications are only simple control logic added to the write path within the PCM chip, without modifying the processor, resulting in a very low deployment threshold. The speedup relationship derived by the authors also allows this benefit to be applied to PCM devices with different parameters.

   Cons: This paper uses fixed write scheduling and reversal thresholds, failing to utilize the data's inherent distribution. Regardless of the actual number of 0 and 1 in a cache line, both phases perform a fixed number of writes. Even if the entire data line is full of 0, the 1-writing phase still completes as usual; however, real memory content is often biased towards 0, and this waste is not uncommon. Reversal is also consistently based on a "more than half" threshold, without considering that writes are performed in rounds. Flipping multiple sets within the same round does not speed things up but instead increases the burden on flags and 0-writing. In other words, the authors criticize the traditional approach as being too conservative, but they only corrected the current estimation, leaving the write count still based on the worst-case scenario.

5. Future Work in My Opinion

   I think that the inversion threshold shouldn't be fixed at "more than half of the 1” but should be dynamically determined based on the data. Writing is done in rounds, and it's only truly effective when flipping reduces the number of rounds required to write 1; if the flipped data still falls within the same round, further flips only increase the number of 0 and the burden on the flags. Therefore, a

dynamic algorithm can be designed to find the minimum number of flips needed to reduce one round before each write operation, based on the actual number of 0 and 1 in the entire data string. This stopping point is the dynamic threshold. Simultaneously, the scheduling should determine the number of rounds based on the actual quantity, skipping phases where there is no data to write.

The inversion direction should also be adjustable. Currently, writing 1 is much slower than writing 0 with current components, so keeping 1 low is correct; however, if future materials reduce the time difference or the current burden of writing 0 becomes heavier, keeping 0 low might be more cost-effective. If the controller can continuously estimate the actual cost of writing 0 and writing 1 based on temperature and component aging during execution, it can automatically decide which side to reverse to; when old data is available, it can also make weighted judgments only on the bits that really need to be changed, taking into account both speed and durability.